

NONPARAMETRIC TECHNIQUES

4.1 INTRODUCTION

In Chapter 3 we treated supervised learning under the assumption that the forms of the underlying density functions were known. Alas, in most pattern recognition applications this assumption is suspect; the common parametric forms rarely fit the densities actually encountered in practice. In particular, all of the classical parametric densities are unimodal (have a single local maximum), whereas many practical problems involve multimodal densities. Furthermore, our hopes are rarely fulfilled that a high-dimensional density might be accurately represented as the product of one-dimensional functions. In this chapter we shall examine *nonparametric* procedures that can be used with arbitrary distributions and without the assumption that the forms of the underlying densities are known.

There are several types of nonparametric methods of interest in pattern recognition. One consists of procedures for estimating the density functions $p(\mathbf{x}|\omega_j)$ from sample patterns. If these estimates are satisfactory, they can be substituted for the true densities when designing the classifier. Another consists of procedures for directly estimating the *a posteriori* probabilities $P(\omega_j|\mathbf{x})$. This is closely related to nonparametric design procedures such as the nearest-neighbor rule, which bypass probability estimation and go directly to decision functions.

4.2 DENSITY ESTIMATION

The basic ideas behind many of the methods of estimating an unknown probability density function are very simple, although rigorous demonstrations that the estimates converge require considerable care. The most fundamental techniques rely on the fact that the probability P that a vector $\mathbf{x}$ will fall in a region $\mathcal{R}$ is given by

$$P = \int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}'. \quad (1)$$

Thus P is a smoothed or averaged version of the density function $p(\mathbf{x})$, and we can estimate this smoothed value of p by estimating the probability P . Suppose that n samples $\mathbf{x}_1, \dots, \mathbf{x}_n$ are drawn independently and identically distributed (i.i.d.) according to the probability law $p(\mathbf{x})$. Clearly, the probability that k of these n fall in $\mathcal{R}$ is given by the binomial law

$$P_k = \binom{n}{k} P^k (1 - P)^{n-k}, \quad (2)$$

and the expected value for k is

$$\mathcal{E}[k] = nP. \quad (3)$$

Moreover, this binomial distribution for k peaks very sharply about the mean, so that we expect that the ratio k/n will be a very good estimate for the probability P , and hence for the smoothed density function. This estimate is especially accurate when n is very large. If we now assume that $p(\mathbf{x})$ is continuous and that the region $\mathcal{R}$ is so small that p does not vary appreciably within it, we can write

$$\int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}' \simeq p(\mathbf{x})V, \quad (4)$$

where $\mathbf{x}$ is a point within $\mathcal{R}$ and V is the volume enclosed by $\mathcal{R}$. Combining Eqs. 1, 3, and 4, we arrive at the following obvious estimate for $p(\mathbf{x})$,

$$p(\mathbf{x}) \simeq \frac{k/n}{V}, \quad (5)$$

as illustrated in Fig. 4.1.

There are several problems that remain—some practical and some theoretical. If we fix the volume V and take more and more training samples, the ratio k/n will

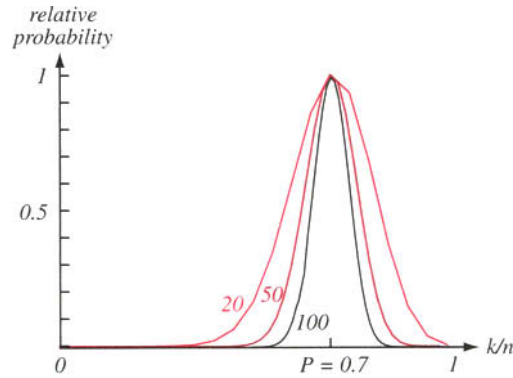


FIGURE 4.1. The relative probability an estimate given by Eq. 4 will yield a particular value for the probability density, here where the true probability was chosen to be 0.7. Each curve is labeled by the total number of patterns n sampled, and is scaled to give the same maximum (at the true probability). The form of each curve is binomial, as given by Eq. 2. For large n , such binomials peak strongly at the true probability. In the limit $n \rightarrow \infty$, the curve approaches a delta function, and we are guaranteed that our estimate will give the true probability.

converge (in probability) as desired, but then we have only obtained an estimate of the space-averaged value of $p(\mathbf{x})$,

$$\frac{P}{V} = \frac{\int_{\mathcal{R}} p(\mathbf{x}') d\mathbf{x}'}{\int_{\mathcal{R}} d\mathbf{x}'} \quad (6)$$

If we want to obtain $p(\mathbf{x})$ rather than just an averaged version of it, we must be prepared to let V approach zero. However, if we fix the number n of samples and let V approach zero, the region will eventually become so small that it will enclose no samples, and our estimate $p(\mathbf{x}) \simeq 0$ will be useless. Or if by chance one or more of the training samples coincide at $\mathbf{x}$, the estimate diverges to infinity, which is equally useless.

From a practical standpoint, we note that the number of samples is always limited. Thus, the volume V cannot be allowed to become arbitrarily small. If this kind of estimate is to be used, one will have to accept a certain amount of variance in the ratio k/n and a certain amount of averaging of the density $p(\mathbf{x})$.

From a theoretical standpoint, it is interesting to ask how these limitations can be circumvented if an unlimited number of samples is available. Suppose we use the following procedure. To estimate the density at $\mathbf{x}$, we form a sequence of regions $\mathcal{R}_1, \mathcal{R}_2, \dots$, containing $\mathbf{x}$ —the first region to be used with one sample, the second with two, and so on. Let V_n be the volume of $\mathcal{R}_n$, k_n be the number of samples falling in $\mathcal{R}_n$, and $p_n(\mathbf{x})$ be the n th estimate for $p(\mathbf{x})$:

$$p_n(\mathbf{x}) = \frac{k_n/n}{V_n} \quad (7)$$

If $p_n(\mathbf{x})$ is to converge to $p(\mathbf{x})$, three conditions appear to be required:

- $\lim_{n \rightarrow \infty} V_n = 0$
- $\lim_{n \rightarrow \infty} k_n = \infty$
- $\lim_{n \rightarrow \infty} k_n/n = 0$.

The first condition assures us that the space averaged P/V will converge to $p(\mathbf{x})$, provided that the regions shrink uniformly and that $p(\cdot)$ is continuous at $\mathbf{x}$. The second condition, which only makes sense if $p(\mathbf{x}) \neq 0$, assures us that the frequency ratio will converge (in probability) to the probability P . The third condition is clearly necessary if $p_n(\mathbf{x})$ given by Eq. 7 is to converge at all. It also says that although a huge number of samples will eventually fall within the small region $\mathcal{R}_n$, they will form a negligibly small fraction of the total number of samples.

There are two common ways of obtaining sequences of regions that satisfy these conditions (Fig. 4.2). One is to shrink an initial region by specifying the volume V_n as some function of n , such as $V_n = 1/\sqrt{n}$. It then must be shown that the random variables k_n and k_n/n behave properly or, more to the point, that $p_n(\mathbf{x})$ converges to $p(\mathbf{x})$. This is basically the Parzen-window method that will be examined in Section 4.3. The second method is to specify k_n as some function of n , such as $k_n = \sqrt{n}$. Here the volume V_n is grown until it encloses k_n neighbors of $\mathbf{x}$. This is the k_n -nearest-neighbor estimation method. Both of these methods do in fact converge, although it is difficult to make meaningful statements about their finite-sample behavior.

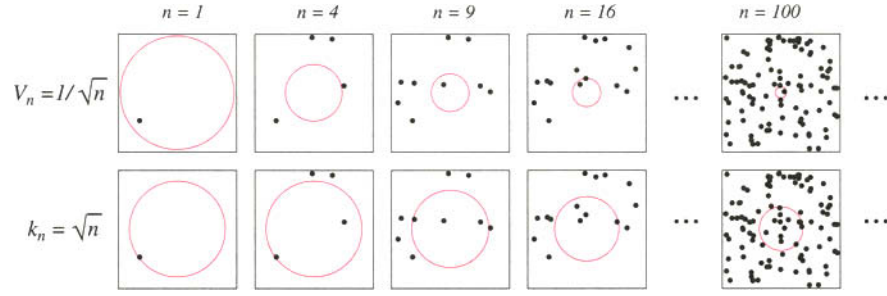


FIGURE 4.2. There are two leading methods for estimating the density at a point, here at the center of each square. The one shown in the top row is to start with a large volume centered on the test point and shrink it according to a function such as $V_n = 1/\sqrt{n}$. The other method, shown in the bottom row, is to decrease the volume in a data-dependent way, for instance letting the volume enclose some number $k_n = \sqrt{n}$ of sample points. The sequences in both cases represent random variables that generally converge and allow the true density at the test point to be calculated.

4.3 PARZEN WINDOWS

The Parzen-window approach to estimating densities can be introduced by temporarily assuming that the region $\mathcal{R}_n$ is a d -dimensional hypercube. If h_n is the length of an edge of that hypercube, then its volume is given by

$$V_n = h_n^d. \quad (8)$$

We can obtain an analytic expression for k_n , the number of samples falling in the hypercube, by defining the following *window function*:

WINDOW
FUNCTION

$$\varphi(\mathbf{u}) = \begin{cases} 1 & |u_j| \leq 1/2; \\ 0 & \text{otherwise.} \end{cases} \quad j = 1, \dots, d \quad (9)$$

Thus, $\varphi(\mathbf{u})$ defines a unit hypercube centered at the origin. It follows that $\varphi((\mathbf{x} - \mathbf{x}_i)/h_n)$ is equal to unity if $\mathbf{x}_i$ falls within the hypercube of volume V_n centered at $\mathbf{x}$, and is zero otherwise. The number of samples in this hypercube is therefore given by

$$k_n = \sum_{i=1}^n \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right), \quad (10)$$

and when we substitute this into Eq. 7 we obtain the estimate

$$p_n(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{V_n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right). \quad (11)$$

This equation suggests a more general approach to estimating density functions. Rather than limiting ourselves to the hypercube window function of Eq. 9, suppose we allow a more general class of window functions. In such a case, Eq. 11 expresses our estimate for $p(\mathbf{x})$ as an average of functions of $\mathbf{x}$ and the samples $\mathbf{x}_i$. In essence, the window function is being used for *interpolation*—each sample contributing to the estimate in accordance with its distance from $\mathbf{x}$.

It is natural to ask that the estimate $p_n(\mathbf{x})$ be a legitimate density function, that is, that it be nonnegative and integrate to one. This can be assured by requiring the window function itself be a density function. To be more precise, if we require that

$$\varphi(\mathbf{x}) \geq 0 \quad (12)$$

and

$$\int \varphi(\mathbf{u}) d\mathbf{u} = 1, \quad (13)$$

and if we maintain the relation $V_n = h_n^d$, then it follows at once that $p_n(\mathbf{x})$ also satisfies these conditions.

Let us examine the effect that the *window width* h_n has on $p_n(\mathbf{x})$. If we define the function $\delta_n(\mathbf{x})$ by

$$\delta_n(\mathbf{x}) = \frac{1}{V_n} \varphi\left(\frac{\mathbf{x}}{h_n}\right), \quad (14)$$

then we can write $p_n(\mathbf{x})$ as the average

$$p_n(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \delta_n(\mathbf{x} - \mathbf{x}_i). \quad (15)$$

Because $V_n = h_n^d$, h_n clearly affects both the amplitude and the width of $\delta_n(\mathbf{x})$ (Fig. 4.3). If h_n is very large, then the amplitude of δ_n is small, and $\mathbf{x}$ must be far from $\mathbf{x}_i$ before $\delta_n(\mathbf{x} - \mathbf{x}_i)$ changes much from $\delta_n(\mathbf{0})$. In this case, $p_n(\mathbf{x})$ is the superposition of n broad, slowly changing functions and is a very smooth “out-of-focus” estimate of $p(\mathbf{x})$. On the other hand, if h_n is very small, then the peak value of $\delta_n(\mathbf{x} - \mathbf{x}_i)$ is large and occurs near $\mathbf{x} = \mathbf{x}_i$. In this case $p(\mathbf{x})$ is the superposition of n sharp pulses centered at the samples—an erratic, “noisy” estimate (Fig. 4.4). For any value of h_n , the distribution is normalized, that is,

$$\int \delta_n(\mathbf{x} - \mathbf{x}_i) d\mathbf{x} = \int \frac{1}{V_n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right) d\mathbf{x} = \int \varphi(\mathbf{u}) d\mathbf{u} = 1. \quad (16)$$

Thus, as h_n approaches zero, $\delta_n(\mathbf{x} - \mathbf{x}_i)$ approaches a Dirac delta function centered at $\mathbf{x}_i$, and $p_n(\mathbf{x})$ approaches a superposition of delta functions centered at the samples.

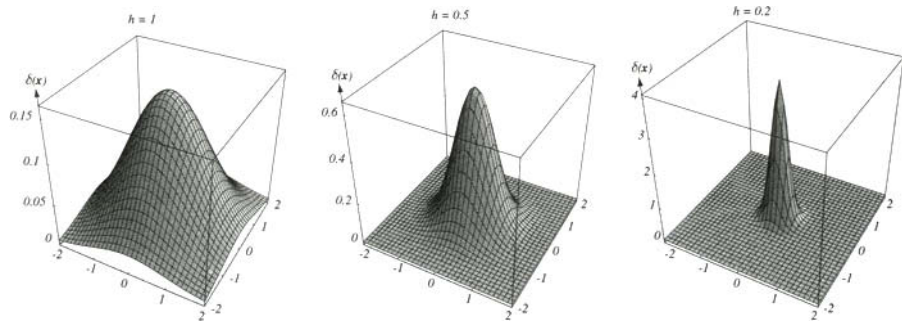


FIGURE 4.4. Examples of two-dimensional circularly symmetric normal Parzen windows for three different values of h . Note that because the $\delta(\mathbf{x})$ are normalized, different vertical scales must be used to show their structure.

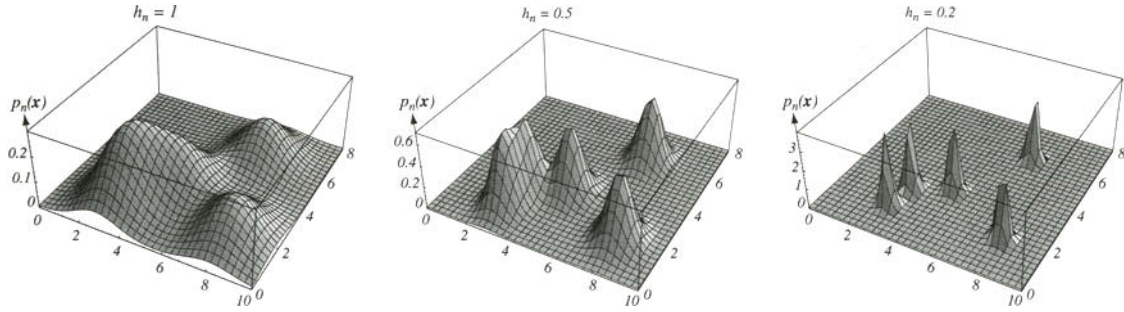


FIGURE 4.4. Three Parzen-window density estimates based on the same set of five samples, using the window functions in Fig. 4.3. As before, the vertical axes have been scaled to show the structure of each distribution.

Clearly, the choice of h_n (or V_n) has an important effect on $p_n(\mathbf{x})$. If V_n is too large, the estimate will suffer from too little resolution; if V_n is too small, the estimate will suffer from too much statistical variability. With a limited number of samples, the best we can do is to seek some acceptable compromise. However, with an unlimited number of samples, it is possible to let V_n slowly approach zero as n increases and have $p_n(\mathbf{x})$ converge to the unknown density $p(\mathbf{x})$.

In discussing convergence, we must recognize that we are talking about the convergence of a sequence of random variables, because for any fixed $\mathbf{x}$ the value of $p_n(\mathbf{x})$ depends on the random samples $\mathbf{x}_1, \dots, \mathbf{x}_n$. Thus, $p_n(\mathbf{x})$ has some mean $\bar{p}_n(\mathbf{x})$ and variance $\sigma_n^2(\mathbf{x})$. We shall say that the estimate $p_n(\mathbf{x})$ converges to $p(\mathbf{x})$ if*

$$\lim_{n \rightarrow \infty} \bar{p}_n(\mathbf{x}) = p(\mathbf{x}) \quad (17)$$

and

$$\lim_{n \rightarrow \infty} \sigma_n^2(\mathbf{x}) = 0. \quad (18)$$

To prove convergence we must place conditions on the unknown density $p(\mathbf{x})$, on the window function $\varphi(\mathbf{u})$, and on the window width h_n . In general, continuity of $p(\cdot)$ at $\mathbf{x}$ is required, and the conditions imposed by Eqs. 12 and 13 are customarily invoked. Below we show that the following additional conditions assure convergence:

$$\sup_{\mathbf{u}} \varphi(\mathbf{u}) < \infty \quad (19)$$

$$\lim_{\|\mathbf{u}\| \rightarrow \infty} \varphi(\mathbf{u}) \prod_{i=1}^d u_i = 0 \quad (20)$$

$$\lim_{n \rightarrow \infty} V_n = 0 \quad (21)$$

and

$$\lim_{n \rightarrow \infty} n V_n = \infty. \quad (22)$$

*This type of convergence is called *convergence in mean square*. Other ways to define the modes of convergence of sequences of random variables are presented in advanced books on probability theory.

Equations 19 and 20 keep $\varphi(\cdot)$ well-behaved, and they are satisfied by most density functions that one might think of using for window functions. Equations 21 and 22 state that the volume V_n must approach zero, but at a rate slower than $1/n$. We shall now see why these are the basic conditions for convergence.

4.3.1 Convergence of the Mean

Consider first $\bar{p}_n(\mathbf{x})$, the mean of $p_n(\mathbf{x})$. Because the samples $\mathbf{x}_i$ are i.i.d. according to the (unknown) density $p(\mathbf{x})$, we have

$$\begin{aligned}\bar{p}_n(\mathbf{x}) &= \mathcal{E}[p_n(\mathbf{x})] \\ &= \frac{1}{n} \sum_{i=1}^n \mathcal{E}\left[\frac{1}{V_n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right)\right] \\ &= \int \frac{1}{V_n} \varphi\left(\frac{\mathbf{x} - \mathbf{v}}{h_n}\right) p(\mathbf{v}) d\mathbf{v} \\ &= \int \delta_n(\mathbf{x} - \mathbf{v}) p(\mathbf{v}) d\mathbf{v}.\end{aligned}\tag{23}$$

CONVOLUTION

This equation shows that the expected value of the estimate is an averaged value of the unknown density—a *convolution* of the unknown density and the window function (see Section A.4.11 of the Appendix). Thus, $\bar{p}_n(\mathbf{x})$ is a blurred version of $p(\mathbf{x})$ as seen through the averaging window. But as V_n approaches zero, $\delta_n(\mathbf{x} - \mathbf{v})$ approaches a delta function centered at $\mathbf{x}$. Thus, if p is continuous at $\mathbf{x}$, Eq. 21 ensures that $\bar{p}_n(\mathbf{x})$ will approach $p(\mathbf{x})$ as n approaches infinity.

4.3.2 Convergence of the Variance

Equation 23 shows that there is no need for an infinite number of samples to make $\bar{p}_n(\mathbf{x})$ approach $p(\mathbf{x})$; one can achieve this for any n merely by letting V_n approach zero. Of course, for a particular set of n samples the resulting “spiky” estimate is useless; this fact highlights the need for us to consider the variance of the estimate. Because $p_n(\mathbf{x})$ is the sum of functions of statistically independent random variables, its variance is the sum of the variances of the separate terms, and hence

$$\begin{aligned}\sigma_n^2(\mathbf{x}) &= \sum_{i=1}^n \mathcal{E}\left[\left(\frac{1}{nV_n} \varphi\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right) - \frac{1}{n} \bar{p}_n(\mathbf{x})\right)^2\right] \\ &= n \mathcal{E}\left[\frac{1}{n^2 V_n^2} \varphi^2\left(\frac{\mathbf{x} - \mathbf{x}_i}{h_n}\right)\right] - \frac{1}{n} \bar{p}_n^2(\mathbf{x}) \\ &= \frac{1}{nV_n} \int \frac{1}{V_n} \varphi^2\left(\frac{\mathbf{x} - \mathbf{v}}{h_n}\right) p(\mathbf{v}) d\mathbf{v} - \frac{1}{n} \bar{p}_n^2(\mathbf{x}).\end{aligned}\tag{24}$$

By dropping the second term, bounding $\varphi(\cdot)$ and using Eq. 23, we obtain

$$\sigma_n^2(\mathbf{x}) \leq \frac{\sup(\varphi(\cdot)) \bar{p}_n(\mathbf{x})}{nV_n}.\tag{25}$$

Clearly, to obtain a small variance we want a large value for V_n , not a small one—a large V_n smoothes out the local variations in density. However, because the numerator stays finite as n approaches infinity, we can let V_n approach zero and still ob-

tain zero variance, provided that nV_n approaches infinity. For example, we can let $V_n = V_1/\sqrt{n}$ or $V_1/\ln n$ or any other function satisfying Eqs. 21 and 22.

This is the principal theoretical result. Unfortunately, it does not tell us how to choose $\varphi(\cdot)$ and V_n to obtain good results in the finite sample case. Indeed, unless we have more knowledge about $p(\mathbf{x})$ than the mere fact that it is continuous, we have no direct basis for optimizing finite sample results.

4.3.3 Illustrations

It is interesting to see how the Parzen window method behaves on some simple examples, and particularly to see the effect of the window function. Consider first the case where $p(\mathbf{x})$ is a zero-mean, unit-variance, univariate normal density. Let the window function be of the same form:

$$\varphi(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}. \quad (26)$$

Finally, let $h_n = h_1/\sqrt{n}$, where h_1 is a parameter at our disposal. Thus $p_n(x)$ is an average of normal densities centered at the samples:

$$p_n(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h_n} \varphi\left(\frac{x - x_i}{h_n}\right). \quad (27)$$

While it is not hard to evaluate Eqs. 23 and 24 to find the mean and variance of $p_n(x)$, it is even more interesting to see numerical results. When a particular set of normally distributed random samples was generated and used to compute $p_n(x)$, the results shown in Fig. 4.5 were obtained. These results depend both on n and h_1 . For $n = 1$, $p_n(x)$ is merely a single Gaussian centered about the first sample, which of course has neither the mean nor the variance of the true distribution. For $n = 10$ and $h_1 = 0.1$ the contributions of the individual samples are clearly discernible; this is not the case for $h_1 = 1$ and $h_1 = 0.5$. As n gets larger, the ability of $p_n(x)$ to resolve variations in $p(x)$ increases. Concomitantly, $p_n(x)$ appears to be more sensitive to local sampling irregularities when n is large, although we are assured that $p_n(x)$ will converge to the smooth normal curve as n goes to infinity. While one should not judge on visual appearance alone, it is clear that many samples are required to obtain an accurate estimate. Figure 4.6 shows analogous results in two dimensions.

As a second one-dimensional example, we let $\varphi(x)$ and h_n be the same as in Fig. 4.5, but let the unknown density be a mixture of a uniform and a triangle density. Figure 4.7 shows the behavior of Parzen-window estimates for this density. As before, the case $n = 1$ tells more about the window function than it tells about the unknown density. For $n = 16$, none of the estimates is particularly good, but results for $n = 256$ and $h_1 = 1$ are beginning to appear acceptable.

4.3.4 Classification Example

In classifiers based on Parzen-window estimation, we estimate the densities for each category and classify a test point by the label corresponding to the maximum posterior. If there are multiple categories with unequal priors we can easily include these too (Problem 4). The decision regions for a Parzen-window classifier depend upon the choice of window function, of course, as illustrated in Fig. 4.8. In general, the training error—the empirical error on the training points themselves—can be made

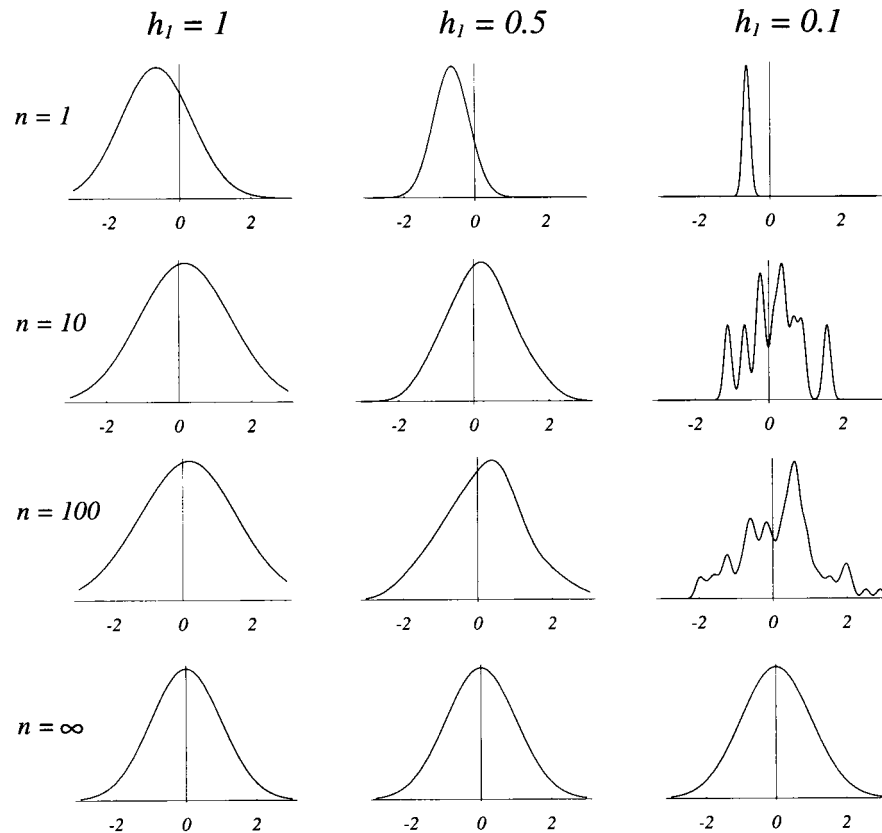


FIGURE 4.5. Parzen-window estimates of a univariate normal density using different window widths and numbers of samples. The vertical axes have been scaled to best show the structure in each graph. Note particularly that the $n = \infty$ estimates are the same (and match the true density function), regardless of window width.

arbitrarily low by making the window width sufficiently small.* However, the goal of creating a classifier is to classify novel patterns, and alas a low training error does not guarantee a small *test* error, as we shall explore in Chapter 9. Although a generic Gaussian window *shape* is plausible, in the absence of other information about the underlying distributions there is little theoretical justification of one window *width* over another.

These density estimation and classification examples illustrate some of the power and some of the limitations of nonparametric methods. Their power resides in their generality. Exactly the same procedure was used for the unimodal normal case and the bimodal mixture case, and we did not need to make any assumptions about the distributions ahead of time. With enough samples, we are essentially assured of convergence to an arbitrarily complicated target density. On the other hand, the number of samples needed may be very large indeed—much greater than would be required if we knew the form of the unknown density. Little or nothing in the way of data reduction is provided, which leads to severe requirements for computation time and storage. Moreover, the demand for a large number of samples grows exponentially

*We ignore cases in which the same feature vector has been assigned to multiple categories.

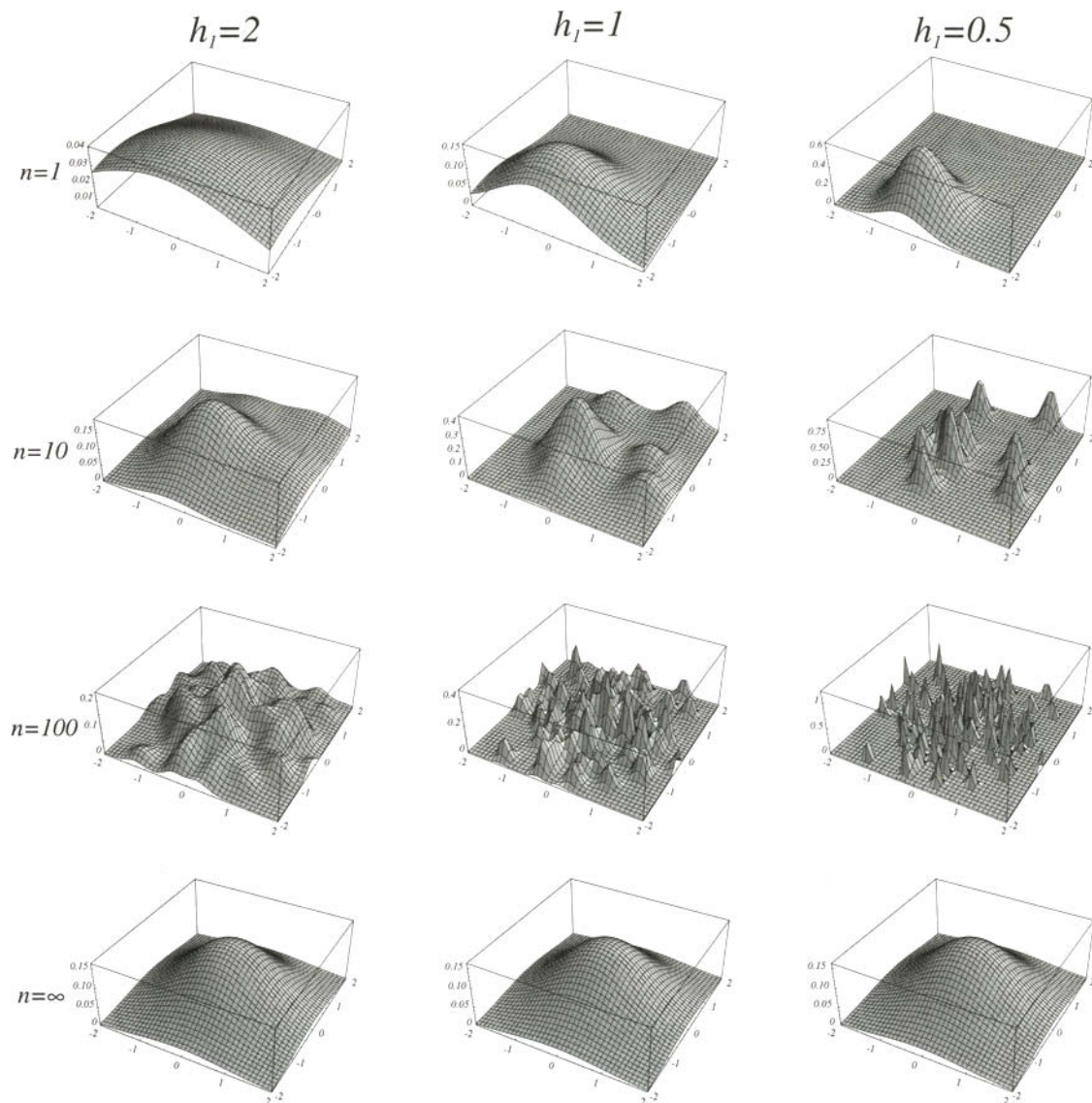


FIGURE 4.6. Parzen-window estimates of a bivariate normal density using different window widths and numbers of samples. The vertical axes have been scaled to best show the structure in each graph. Note particularly that the $n = \infty$ estimates are the same (and match the true distribution), regardless of window width.

with the dimensionality of the feature space. This limitation is called the “curse of dimensionality,” and severely restricts the practical application of such nonparametric procedures (Problem 11). The fundamental reason for the curse of dimensionality is that high-dimensional functions have the potential to be much more complicated than low-dimensional ones, and that those complications are harder to discern. The only way to beat the curse is to incorporate knowledge about the data that is *correct*.

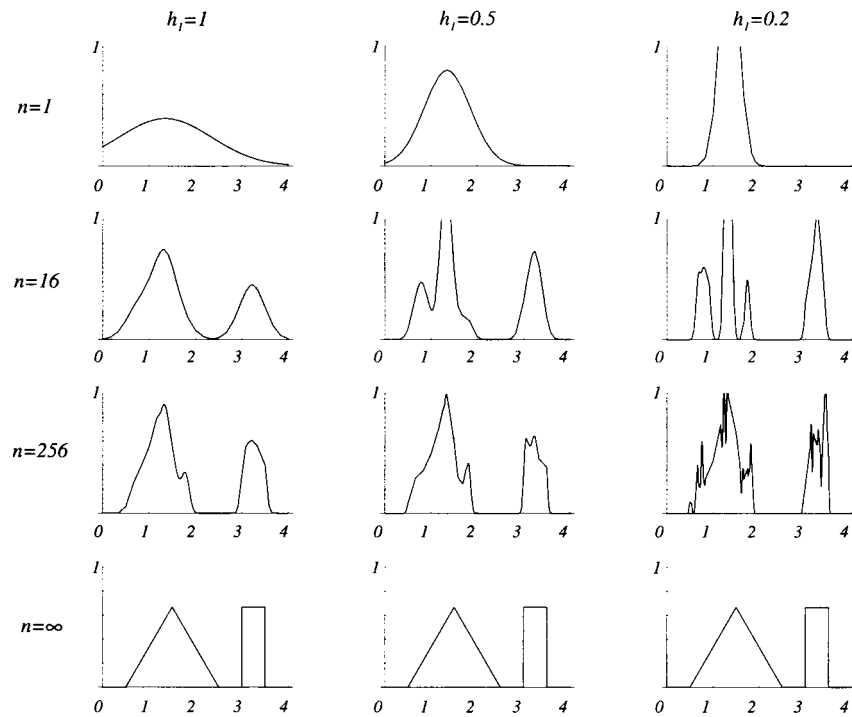


FIGURE 4.7. Parzen-window estimates of a bimodal distribution using different window widths and numbers of samples. Note particularly that the $n = \infty$ estimates are the same (and match the true distribution), regardless of window width.

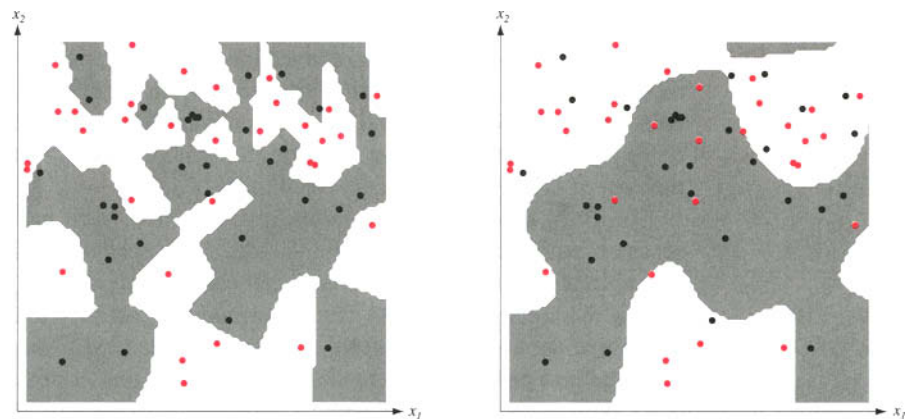


FIGURE 4.8. The decision boundaries in a two-dimensional Parzen-window dichotomizer depend on the window width h . At the left a small h leads to boundaries that are more complicated than for large h on same data set, shown at the right. Apparently, for these data a small h would be appropriate for the upper region, while a large h would be appropriate for the lower region; no single window width is ideal overall.

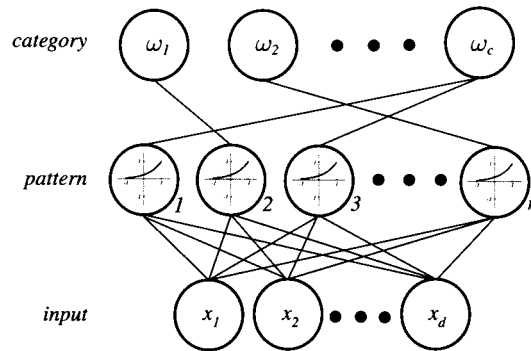


FIGURE 4.9. A probabilistic neural network (PNN) consists of d input units, n pattern units, and c category units. Each pattern unit forms the inner product of its weight vector and the normalized pattern vector $\mathbf{x}$ to form $z = \mathbf{w}^t \mathbf{x}$, and then it emits $\exp[(z - 1)/\sigma^2]$. Each category unit sums such contributions from the pattern unit connected to it. This ensures that the activity in each of the category units represents the Parzen-window density estimate using a circularly symmetric Gaussian window of covariance $\sigma^2 \mathbf{I}$, where $\mathbf{I}$ is the d -by- d identity matrix.

4.3.5 Probabilistic Neural Networks (PNNs)

Most pattern recognition methods can be implemented in a parallel fashion that trades space complexity for time complexity. These implementations are naturally represented as artificial neural networks, a topic that we treat in detail in Chapter 6. As a preview, we can take this opportunity to show how the Parzen-window method can be implemented as a neural network known as a Probabilistic Neural Network (Fig. 4.9). Suppose we wish to form a Parzen estimate based on n patterns, each of which is d -dimensional, randomly sampled from c classes. The PNN for this case consists of d input units comprising the input layer, where each unit is connected to each of the n pattern units; each pattern unit is, in turn, connected to one and only one of the c category units. The connections from the input to pattern units represent modifiable weights, which will be trained. (While these weights are merely parameters and could be represented by a vector $\boldsymbol{\theta}$, in keeping with the established terminology in neural networks we shall use the symbol $\mathbf{w}$.) Each category unit computes the sum of the pattern units connected to it.

The PNN is trained in the following way. First, each pattern $\mathbf{x}$ of the training set is normalized to have unit length—that is, is scaled so that $\sum_{i=1}^d x_i^2 = 1$. The first normalized training pattern is placed on the input units. The modifiable weights linking the input units and the first pattern unit are set such that $\mathbf{w}_1 = \mathbf{x}_1$. (Note that because of the normalization of $\mathbf{x}_1$, $\mathbf{w}_1$ is normalized too.) Then, a single connection from the first pattern unit is made to the category unit corresponding to the known class of that pattern. The process is repeated with each of the remaining training patterns, setting the weights to the successive pattern units such that $\mathbf{w}_k = \mathbf{x}_k$ for $k = 1, 2, \dots, n$. After such training we have a network that is fully connected between input and pattern units, and sparsely connected from pattern to category units. If we denote the components of the j th pattern as x_{jk} and the weights to the j th pattern unit w_{jk} , for $j = 1, 2, \dots, n$ and $k = 1, 2, \dots, d$, then our algorithm is as follows:

INPUT UNIT
PATTERN UNIT
CATEGORY UNIT
WEIGHT

Algorithm 1. (PNN training)

```

1 begin initialize  $j \leftarrow 0, n, a_{ji} \leftarrow 0$  for  $j = 1, \dots, n; i = 1, \dots, c$ 
2   do  $j \leftarrow j + 1$ 
3      $x_{jk} \leftarrow x_{jk} / \left( \sum_i x_{ji}^2 \right)^{1/2}$  (normalize)
4      $w_{jk} \leftarrow x_{jk}$  (train)
5     if  $\mathbf{x}_j \in \omega_i$  then  $a_{ji} \leftarrow 1$ 
6   until  $j = n$ 
7 end

```

NET ACTIVATION

The trained network is then used for classification in the following way. A normalized test pattern $\mathbf{x}$ is placed at the input units. Each pattern unit computes the inner product to yield the *net activation* or simply *net*,

$$net_k = \mathbf{w}_k^t \mathbf{x}, \quad (28)$$

ACTIVATION FUNCTION

and emits a nonlinear function of net_k ; each category unit sums the contributions from all pattern units connected to it. The nonlinear function is $e^{(net_k-1)/\sigma^2}$, where σ is a parameter set by the user and determines the width of the effective Gaussian window. This *activation function* or transfer function, here must be an exponential to implement the Parzen windows algorithm. To see this, consider an (unnormalized) Gaussian window centered on the position of one of the training patterns $\mathbf{w}_k$. We work backwards from the desired Gaussian window function to infer the nonlinear activation function that should be employed by the pattern units. That is, if we let our effective width h_n be a constant, the window function is

$$\begin{aligned}
 \varphi\left(\frac{\mathbf{x} - \mathbf{w}_k}{h_n}\right) &\overset{\text{desired Gaussian}}{\propto} e^{-(\mathbf{x} - \mathbf{w}_k)^t (\mathbf{x} - \mathbf{w}_k) / 2\sigma^2} \\
 &= e^{-(\mathbf{x}'\mathbf{x} + \mathbf{w}_k^t \mathbf{w}_k - 2\mathbf{x}'\mathbf{w}_k) / 2\sigma^2} = \underbrace{e^{(net_k - 1) / \sigma^2}}_{\text{activation function}}, \quad (29)
 \end{aligned}$$

where we have used our normalization conditions $\mathbf{x}'\mathbf{x} = \mathbf{w}_k^t \mathbf{w}_k = 1$. Thus each pattern unit contributes to its associated category unit a signal equal to the probability the test point was generated by a Gaussian centered on the associated training point. The sum of these local estimates (computed at the corresponding category unit) gives the discriminant function $g_i(\mathbf{x})$ —the Parzen-window estimate of the underlying distribution. The $\max_i g_i(\mathbf{x})$ operation gives the desired category for the test point (Algorithm 2).

Algorithm 2. (PNN Classification)

```

1 begin initialize  $k \leftarrow 0, \mathbf{x} \leftarrow$  test pattern
2   do  $k \leftarrow k + 1$ 
3      $net_k \leftarrow \mathbf{w}_k^t \mathbf{x}$ 

```

```

4      if  $a_{ki} = 1$  then  $g_i \leftarrow g_i + \exp[(net_k - 1)/\sigma^2]$ 
5      until  $k = n$ 
6      return  $class \leftarrow \arg \max_i g_i(\mathbf{x})$ 
7 end

```

One of the benefits of PNNs is their speed of learning, because the learning rule (i.e., setting $\mathbf{w}_k = \mathbf{x}_k$) is simple and requires only a single pass through the training data. The space complexity (amount of memory) for the PNN is easy to determine by counting the number of connections in Fig. 4.9— $O((n+1)d)$. This can be quite severe for instance in a hardware application, because both n and d can be quite large. The time complexity for classification by the parallel implementation of Fig. 4.9 is $O(1)$, since the n inner products of Eq. 28 can be done in parallel. Thus this PNN architecture could find uses where recognition speed is important and storage is not a severe limitation. Another benefit is that new training patterns can be incorporated into a previously trained classifier quite easily; this might be important for a particular on-line application.

4.3.6 Choosing the Window Function

As we have seen, one of the problems encountered in the Parzen-window/PNN approach concerns the choice of the sequence of cell-volume sizes $V_1, V_2, \dots$ or overall window size (or indeed other window parameters, such as shape or orientation). For example, if we take $V_n = V_1/\sqrt{n}$, the results for any finite n will be very sensitive to the choice for the initial volume V_1 . If V_1 is too small, most of the volumes will be empty, and the estimate $p_n(\mathbf{x})$ will be very erratic (Fig. 4.7). On the other hand, if V_1 is too large, important spatial variations in $p(\mathbf{x})$ may be lost due to averaging over the cell volume. Furthermore, it may well be the case that a cell volume appropriate for one region of the feature space might be entirely unsuitable in a different region (Fig. 4.8). In Chapter 9 we shall consider general methods, including cross-validation, which are often used in conjunction with Parzen windows. In brief, cross-validation requires taking some small portion of the data to form a *validation set*. The classifier is trained on the remaining patterns in the training set, but the window width is adjusted to give the smallest error on the validation set.

4.4 k_n -NEAREST-NEIGHBOR ESTIMATION

PROTOTYPES

A potential remedy for the problem of the unknown “best” window function is to let the cell volume be a function of the *training data*, rather than some arbitrary function of the overall number of samples. For example, to estimate $p(\mathbf{x})$ from n training samples or *prototypes* we can center a cell about $\mathbf{x}$ and let it grow until it captures k_n samples, where k_n is some specified function of n . These samples are the k_n *nearest-neighbors* of $\mathbf{x}$. If the density is high near $\mathbf{x}$, the cell will be relatively small, which leads to good resolution. If the density is low, it is true that the cell will grow large, but it will stop soon after it enters regions of higher density. In either case, if we take

$$p_n(\mathbf{x}) = \frac{k_n/n}{V_n} \quad (30)$$